

INFORME DE ACTIVIDAD PRÁCTICA

CERTIFICADOS SSL

Del sitio sin cifrado al certificado de Let's Encrypt instalado a mano

Elemento	Detalle
Autor	Juan Sebastián Pineda Santafé
Aplicación	Taller de Cifrado Clásico con módulo de inicio de sesión
Dominio	santafe-pineda.shop
Servidor web	nginx como proxy inverso y terminador TLS
Sistema operativo	Rocky Linux 9
Certificados	Autofirmado (OpenSSL) y Let's Encrypt (certbot, modo manual)

1. Introducción y objetivo

La aplicación web desarrollada en los talleres anteriores permite cifrar, descifrar y analizar criptogramas con los métodos de César, Afín y Vigenère, y exige un inicio de sesión para llegar a esas herramientas. Ese inicio de sesión resolvió quién puede entrar, pero dejó abierta una cuestión distinta y anterior: por dónde viajan las credenciales. Mientras el sitio se sirva por HTTP, el nombre de usuario y la contraseña recorren la red en texto plano, y cualquiera con acceso al camino —una red inalámbrica compartida, un conmutador intermedio, el propio proveedor— puede leerlos sin esfuerzo ni herramientas especiales.

El objetivo de esta práctica es cerrar esa brecha protegiendo la información en tránsito mediante certificados SSL/TLS. El trabajo recorre deliberadamente tres estados del mismo sitio, en este orden: primero sin ningún certificado, para dejar constancia del riesgo; después con un certificado autofirmado, que cifra correctamente pero no consigue que ningún navegador confíe en él; y por último con un certificado gratuito de Let's Encrypt, emitido e instalado de forma manual, que sí es aceptado por los clientes y permite forzar que toda la navegación ocurra sobre HTTPS.

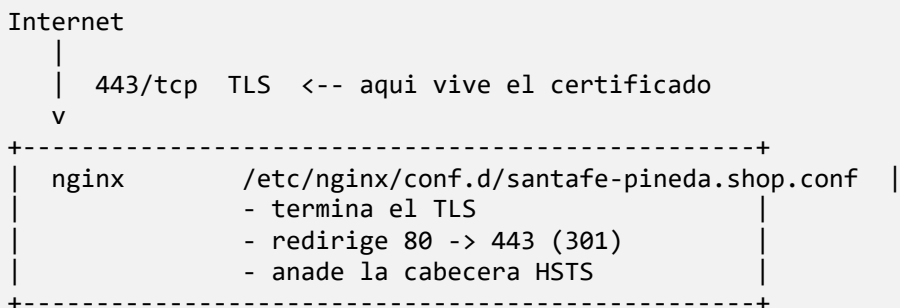
La distinción entre las etapas segunda y tercera es el núcleo del ejercicio, porque ilustra que **cifrar y autenticar son dos problemas separados**. El certificado autofirmado resuelve el primero con la misma solidez criptográfica que cualquier otro: el canal queda igualmente protegido. Lo que no resuelve es el segundo, porque nadie ajeno al servidor respalda la afirmación de que ese servidor es quien dice ser. El certificado de Let's Encrypt aporta exactamente esa pieza que falta.

2. Entorno y arquitectura del despliegue

La aplicación está construida con Angular 22 en modo de renderizado del lado del servidor sobre Node.js 24 y Express 5. Ese proceso escucha únicamente en la dirección de bucle local, en el puerto 4000, y no es accesible desde internet. Delante de él se sitúa nginx, que es quien recibe las conexiones públicas en los puertos 80 y 443, termina el TLS y reenvía las peticiones ya descifradas por el bucle local.

Esta separación tiene una consecuencia práctica que conviene enunciar desde el principio: el certificado se instala en nginx, no en la aplicación. La aplicación nunca ve una conexión cifrada. Por eso, como se detalla en el apartado 9, hubo que indicarle explícitamente que confíe en las cabeceras que nginx añade para informarle del protocolo original; de lo contrario la aplicación creería que todo su tráfico llega en claro y se comportaría en consecuencia.

Arquitectura del despliegue



```

|
| http 127.0.0.1:4000 (bucle local, sin cifrar)
v
+-----+
| Aplicacion Angular SSR + Express (systemd) |
| X-Forwarded-Proto: https --> req.secure    |
+-----+

```

El dominio `santafe-pineda.shop` apunta con un registro A a la dirección IP pública del servidor. Ese detalle no es accesorio: Let's Encrypt comprueba el control sobre el dominio consultando el DNS público, de modo que sin esa correspondencia la emisión del certificado es imposible.

3. Etapa 1: el sitio sin certificado

El punto de partida es la aplicación publicada tal y como quedó en el taller anterior, respondiendo sólo por HTTP en el puerto 80. La configuración de nginx en esta etapa se limita a reenviar las peticiones a la aplicación, sin ninguna referencia a TLS.

```

/etc/nginx/conf.d/santafe-pineda.shop.conf (etapa 1)
server {
    listen      80;
    listen      [::]:80;
    server_name santafe-pineda.shop www.santafe-pineda.shop;

    location / {
        include    /etc/nginx/fragmentos/proxy-node.conf;
        proxy_pass http://127.0.0.1:4000;
    }
}

```

Firefox marca el sitio con un candado tachado y, al situar el cursor sobre el campo de contraseña del formulario, muestra el aviso de que la conexión no es segura. Esa advertencia es la manifestación visible de un problema que se puede comprobar de forma mucho más directa capturando el tráfico mientras se inicia sesión:

```

Captura del tráfico durante un inicio de sesión
sudo tcpdump -i any -A -s0 'tcp port 80 and host santafe-pineda.shop'

POST /api/auth/login HTTP/1.1
Host: santafe-pineda.shop
Content-Type: application/json

{"username":"admin","password":"MiClaveSecreta123"}

```

La contraseña aparece íntegra en la captura. Conviene subrayar lo que esto significa a la luz del taller anterior: allí se almacenaron las contraseñas con BCrypt precisamente para que ni siquiera el

administrador de la base de datos pudiera leerlas, y se construyó un control de intentos fallidos para frenar la fuerza bruta. Todo ese trabajo **protege la contraseña en reposo, pero no hace nada por ella mientras viaja**. Un atacante situado en la red no necesita adivinar ni romper el hash: le basta con escuchar. El certificado SSL es lo que cierra esa vía.

Una prueba con SSL Labs en esta etapa no produce calificación alguna, sino el mensaje "Assessment failed: No secure protocols supported", que es la forma que tiene la herramienta de decir que no hay nada que evaluar porque el puerto 443 ni siquiera responde.

INSERTAR CAPTURA · 01-sin-ssl-firefox.png

Firefox en <http://santafe-pineda.shop/login> con el candado tachado y el aviso "Esta conexión no es segura" sobre el campo de contraseña.

INSERTAR CAPTURA · 02-sin-ssl-captura-red.png

Salida de tcpdump durante un inicio de sesión: el usuario y la contraseña legibles en texto plano.

INSERTAR CAPTURA · 03-sin-ssl-ssllabs.png

SSL Labs sobre el dominio: "Assessment failed: No secure protocols supported".

4. Etapa 2: certificado autofirmado

El certificado autofirmado se generó con OpenSSL siguiendo el procedimiento indicado en el enunciado, adaptado de Apache a nginx. Son tres operaciones encadenadas y conviene verlas por separado, porque es en la tercera donde reside toda la diferencia con un certificado real.

4.1. Paso 1: la llave privada

Primero se genera el par de claves RSA de 2048 bits. El archivo resultante contiene la llave privada y es el secreto que sostiene toda la seguridad del sitio: quien lo obtenga puede suplantar al servidor y descifrar el tráfico que se haya grabado. Por eso se le restringen los permisos a lectura y escritura sólo para el superusuario.

```
openssl genrsa -out /etc/pki/tls/private/santafe-pineda.shop.key 2048
chmod 600 /etc/pki/tls/private/santafe-pineda.shop.key
```

4.2. Paso 2: la solicitud de firma

A continuación se construye una solicitud de firma de certificado, o CSR. Es un documento que contiene la clave pública y los datos del titular, y que en un escenario real se enviaría a una Autoridad Certificadora para que lo firmase.

```
openssl req -new \
    -key /etc/pki/tls/private/santafe-pineda.shop.key \
    -out /etc/pki/tls/certs/santafe-pineda.shop.csr \
    -subj "/C=CO/ST=Santerder/L=Bucaramanga/O=Programa de Ingenieria de
    Sistemas/OU=Seguridad de la Informacion/CN=santafe-pineda.shop"
```

```
rocky@3.134.58.85: Permission denied (publickey).
PS C:\Users\js020\downloads> ssh -i "C:\Users\js020\Downloads\huesoperro.pem" ubuntu@3.134.58.85
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 7.0.0-1012-aws x86_64)
```

```
* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:        https://ubuntu.com/pro
```

System information as of Sun Sep 20 20:04:51 UTC 2026

```
System load:  0.08      Temperature:   -273.1 C
Usage of /:    79.3% of 6.71GB   Processes:    122
Memory usage:  34%      Users logged in: 0
Swap usage:    0%        IPv4 address for ens5: 172.31.22.193
```

* Ubuntu Pro delivers the most comprehensive open source security and compliance features.

<https://ubuntu.com/aws/pro>

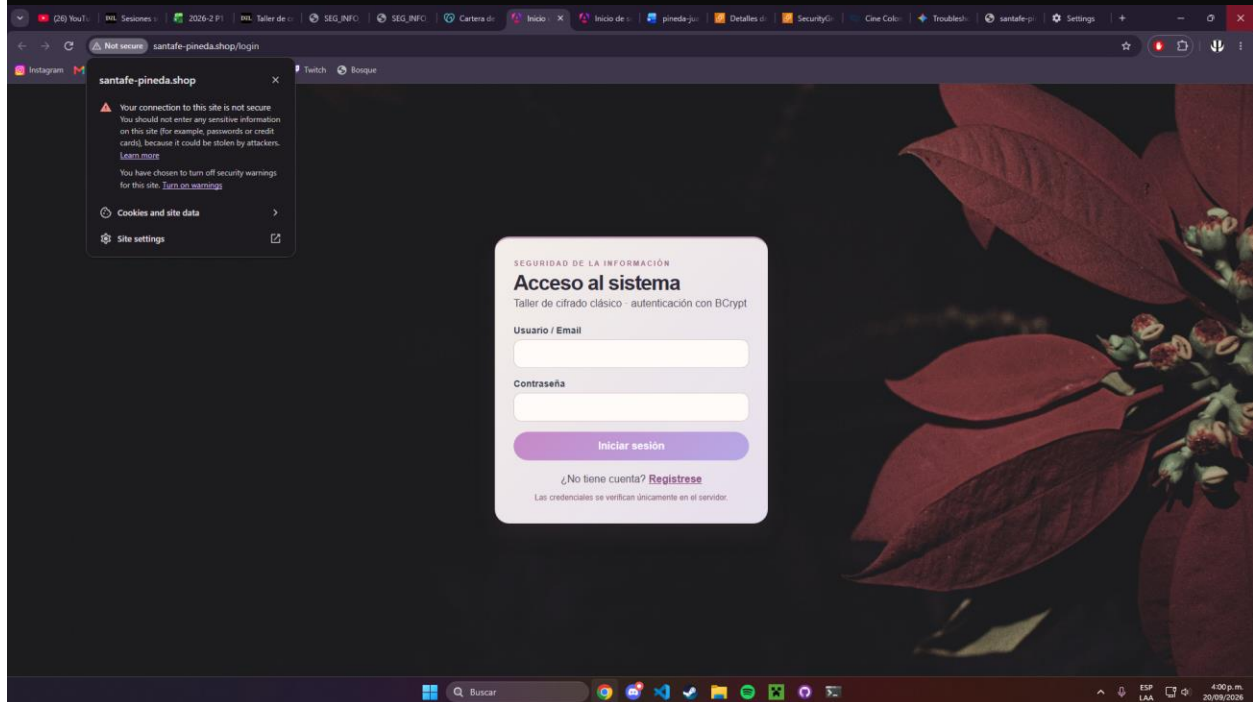
Expanded Security Maintenance for Applications is not enabled.

6 updates can be applied immediately.
To see these additional updates run: `apt list --upgradable`

Enable ESM Apps to receive additional future security updates.
See <https://ubuntu.com/esm> or run: `sudo pro status`

The list of available updates is more than a week old.
To check for new updates run: `sudo apt update`

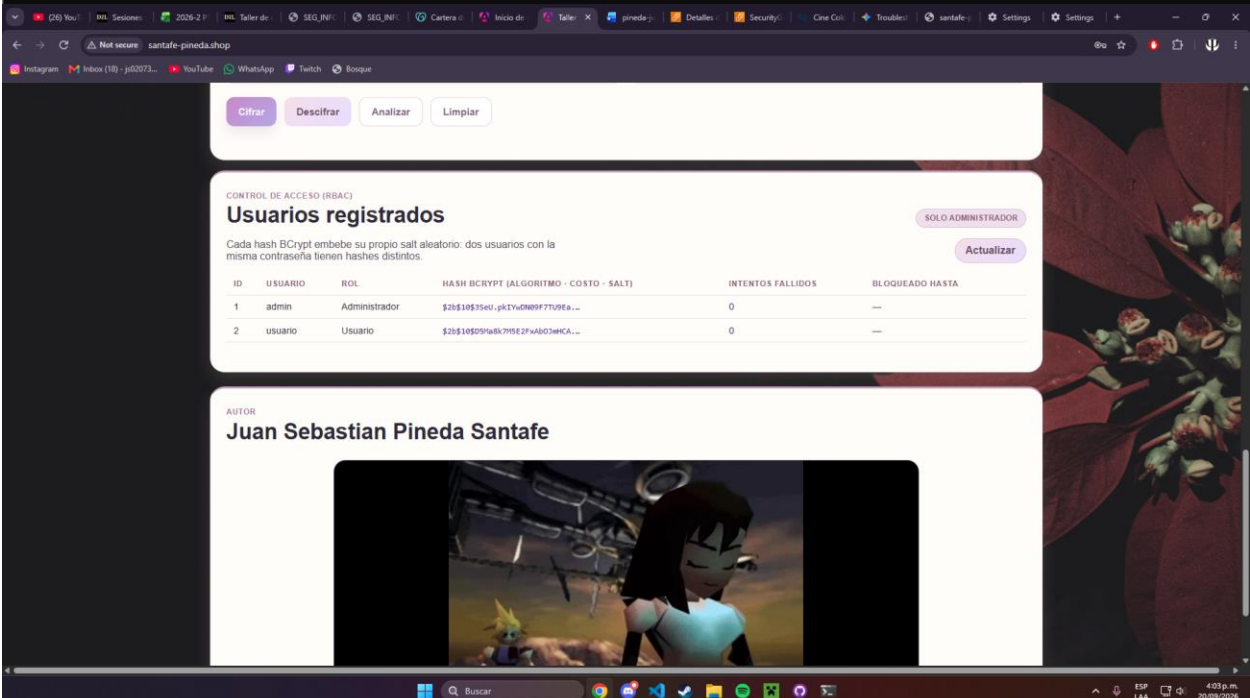
Last login: Thu Sep 3 22:41:59 2026 from 186.84.20.202
ubuntu@ip-172-31-22-193:~\$ |



```
ubuntu@ip-172-31-22-193:~/Taller-de-Cifrado-Clasico/Taller/Taller-cifrado$ cd ~/Taller-de-Cifrado-Clasico/Taller/Taller-cifrado
git pull
sed -i '/^FORZAR_HTTPS=/d; /^COOKIE_SECURE=/d' .env
sudo sed -i '/^FORZAR_HTTPS=/d; /^COOKIE_SECURE=/d' /opt/taller-cifrado/.env
sudo cp deploy/systemd/taller-cifrado.service /etc/systemd/system/
sudo systemctl daemon-reload && sudo systemctl restart taller-cifrado
curl -I http://santafe-pineda.shop/login
remote: Enumerating objects: 19, done.
remote: Counting objects: 100% (19/19), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 10 (delta 7), reused 10 (delta 7), pack-reused 0 (from 0)
Unpacking objects: 100% (10/10), 2.06 KiB | 210.00 KiB/s, done.
From https://github.com/SemiramyZ/Taller-de-Cifrado-Clasico
   ec463ec..e9b9231  main      -> origin/main
Updating ec463ec..e9b9231
Fast-forward
  Taller/Taller-cifrado/.env.example | 12 ++++++-----
  Taller/Taller-cifrado/deploy/systemd/taller-cifrado.service | 10 ++++++++--
  Taller/Taller-cifrado/docs/PASO-A-PASO-AWS.md | 7 ++++++-
  3 files changed, 22 insertions(+), 7 deletions(-)
HTTP/1.1 502 Bad Gateway
Server: nginx/1.24.0 (Ubuntu)
Date: Sun, 20 Sep 2026 20:59:09 GMT
Content-Type: text/html
Content-Length: 166
Connection: keep-alive

ubuntu@ip-172-31-22-193:~/Taller-de-Cifrado-Clasico/Taller/Taller-cifrado$ curl -I http://santafe-pineda.shop/login
HTTP/1.1 200 OK
Server: nginx/1.24.0 (Ubuntu)
Date: Sun, 20 Sep 2026 20:59:14 GMT
Content-Type: text/html; charset=UTF-8
Connection: keep-alive
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
Referrer-Policy: strict-origin-when-cross-origin

ubuntu@ip-172-31-22-193:~/Taller-de-Cifrado-Clasico/Taller/Taller-cifrado$ |
```



The screenshot shows a web browser displaying the 'santafe-pineda.shop' website. The page has a dark theme with a red and green floral pattern on the right side. At the top, there are four buttons: 'Cifrar', 'Descifrar', 'Analizar', and 'Limpiar'. Below these is a section titled 'CONTROL DE ACCESO (RBAC) Usuarios registrados'. It contains a table with two users: 'admin' (Administrator) and 'usuario' (User). Both users have a 'HASH BCrypt' and '0' failed attempts. There are buttons for 'SOLO ADMINISTRADOR' and 'Actualizar'. Below the table, there is a section for the author 'Juan Sebastian Pineda Santafe' with a video thumbnail showing a character from an anime.

ID	USUARIO	ROL	HASH BCrypt (ALGORITMO · COSTO · SALT)	INTENTOS FALLIDOS	BLOQUEADO HASTA
1	admin	Administrador	\$2b\$10\$5t5eU_gkZv0N09F7T09EA...	0	---
2	usuario	Usuario	\$2b\$10\$09nabK7H8E2FvAB03erCA...	0	---

Qualys. SSL Labs

Home Projects Qualys Free Trial Contact

You are here: [Home](#) > [Projects](#) > [SSL Server Test](#) > santafe-pineda.shop

SSL Report: santafe-pineda.shop (3.134.58.85)

Assessed on: Sun, 20 Sep 2026 21:04:33 UTC | [Hide](#) | [Clear cache](#) [Scan Another »](#)

Assessment failed: Unable to connect to the server

Known Problems

There are some errors that we cannot fix properly in the current version. They will be addressed in the next generation version, which is currently being developed.

- No secure protocols supported** - if you get this message, but you know that the site supports SSL, wait until the cache expires on its own, then try again, making sure the hostname you enter uses the "www" prefix (e.g., "www.sslabs.com", not just "sslabs.com")
- no more data allowed for version 1 certificate** - the certificate is invalid. It is declared as version 1, but uses extensions, which were introduced in version 3. Browsers might ignore this problem, but our parser is strict and refuses to proceed. We'll try to find a different parser to avoid this problem
- Failed to obtain certificate and Internal Error** - errors of this type will often be reported for servers that use connection rate limits or block connections in response to unusual traffic. Problems of this type are very difficult to diagnose. If you have access to the server being tested, before reporting a problem to us, please check that there is no rate limiting or IDS in place.
- NetScaler issues** - some NetScaler versions appear to reject SSL handshakes that do not include certain suites or handshakes that use a few suites. If the test is failing and there is a NetScaler load balancer in place, that's most likely the reason.
- Unexpected failure** - our tests are designed to fail when unusual results are observed. This usually happens when there are multiple TLS servers behind the same IP address. In such cases we can't provide accurate results, which is why we fail

Common Error Messages

- Connect timed out** - server did not respond to our connection request, sometimes before we are dynamically blocked when our tests are detected
- No route to host** - unable to reach the server
- Unable to connect to server** - failed to connect to the server, it usually happens due to firewall restrictions
- Connection reset** - we got disconnected from the server
- Unrecognized SSL message, plaintext connection?** - the server responded with plain-text HTTP on HTTPS port
- Received fatal alert: handshake_failure** - this is either a faulty SSL server or some other server listening on port 443; if the SSL version of the web site works in your browser, please report this issue to us
- Failed to communicate with the secure server** - No secure protocol supported. Possibly this server only supports a draft version of TLS 1.3

SSL Report v2.4.3

Copyright © 2009-2026 Qualys, Inc. All Rights Reserved [Privacy Policy](#) [Terms and Conditions](#)

```
ubuntu@ip-172-31-22-193:~/Taller-de-Cifrado-Clasico/Taller/Taller-cifrado$ sudo tcpdump -r /tmp/captur
a.pcap -A | grep -A25 'POST /api/auth/login'
reading from file /tmp/captura.pcap, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144
Warning: interface names might be incorrect
21:13:10.591307 ens5 In IP dynamic-ip-1868420202.cable.net.co.18443 > ip-172-31-22-193.us-east-2.com
pute.internal.http: Flags [P.], seq 927:1428, ack 818, win 252, length 501: HTTP: POST /api/auth/logi
HTTP/1.1
E.....@.v...<.T.....H..P.O.a...P.....POST /api/auth/login HTTP/1.1
Host: santafe-pineda.shop
Connection: keep-alive
Content-Length: 44
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/15
3.0.0.0 Safari/537.36
Accept: application/json, text/plain, */*
Content-Type: application/json
Origin: http://santafe-pineda.shop
Referer: http://santafe-pineda.shop/login
Accept-Encoding: gzip, deflate
Accept-Language: es-ES,es;q=0.9,en;q=0.8

{"username":"admin","password":"Admin2026*"}
21:13:10.632028 ens5 Out IP ip-172-31-22-193.us-east-2.compute.internal.http > dynamic-ip-1868420202.
cable.net.co.18443: Flags [.], ack 1428, win 481, length 0
E..(..@. ....T...PH.....O.VP.....
21:13:10.655420 ens5 Out IP ip-172-31-22-193.us-east-2.compute.internal.http > dynamic-ip-1868420202.
cable.net.co.18443: Flags [P.], seq 818:1347, ack 1428, win 481, length 529: HTTP: HTTP/1.1 200 OK
E..9..@. ....T...PH.....O.VP....*.HTTP/1.1 200 OK
Server: nginx/1.24.0 (Ubuntu)
Date: Sun, 20 Sep 2026 21:13:10 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 54
Connection: keep-alive
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
Referrer-Policy: strict-origin-when-cross-origin
Set-Cookie: sid=1-4WPSoIkSmZ60WNM18a-4jsRGnwT5CIrqMx_EpMqvo; Max-Age=7200; Path=/; Expires=Sun, 20 Sep
2026 23:13:10 GMT; HttpOnly; SameSite=Strict
ubuntu@ip-172-31-22-193:~/Taller-de-Cifrado-Clasico/Taller/Taller-cifrado$ |
```

```

ubuntu@ip-172-31-22-193:~/Taller-de-Cifrado-Clasico/Taller/Taller-cifrado$ sudo openssl x509 -in /etc/
ssl/certs/santafe-pineda.shop.crt -noout \
    -subject -issuer -dates -ext subjectAltName
subject=C = CO, ST = Santander, L = Bucaramanga, O = Programa de Ingenieria de Sistemas, OU = Seguridad
d de la Informacion, CN = santafe-pineda.shop
issuer=C = CO, ST = Santander, L = Bucaramanga, O = Programa de Ingenieria de Sistemas, OU = Seguridad
de la Informacion, CN = santafe-pineda.shop
notBefore=Sep 20 21:17:28 2026 GMT
notAfter=Dec 23 21:17:28 2028 GMT
X509v3 Subject Alternative Name:
    DNS:santafe-pineda.shop, DNS:www.santafe-pineda.shop
ubuntu@ip-172-31-22-193:~/Taller-de-Cifrado-Clasico/Taller/Taller-cifrado$
==> 6. Verificacion del certificado generado
subject=C = CO, ST = Santander, L = Bucaramanga, O = Programa de Ingenieria de Sistemas, OU = Seguridad
d de la Informacion, CN = santafe-pineda.shop
issuer=C = CO, ST = Santander, L = Bucaramanga, O = Programa de Ingenieria de Sistemas, OU = Seguridad
de la Informacion, CN = santafe-pineda.shop
notBefore=Sep 20 21:17:28 2026 GMT
notAfter=Dec 23 21:17:28 2028 GMT
serial=75CCE3E501247DFA0ACF95DD86F7FE2988212570
X509v3 Subject Alternative Name:
    DNS:santafe-pineda.shop, DNS:www.santafe-pineda.shop

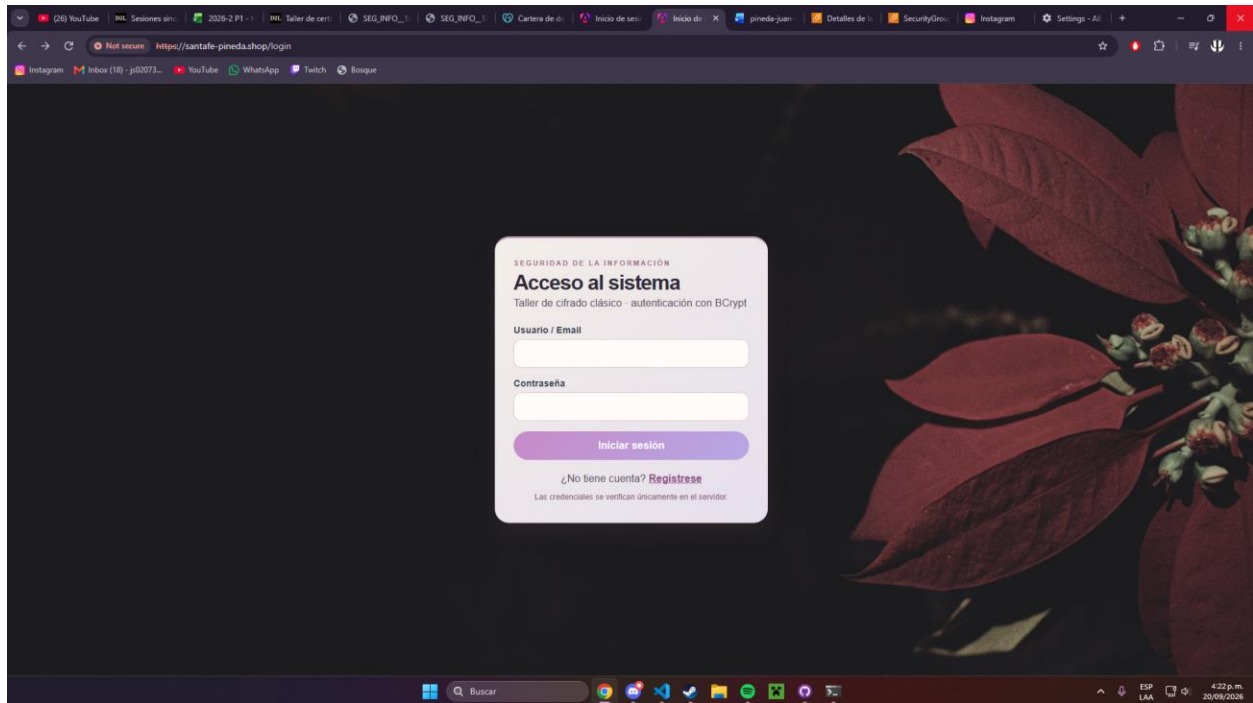
Emisor y titular son el mismo: esa es la 'problematica asociada'.
==> 7. La aplicacion pasa a modo HTTPS
    Aplicacion reiniciada en modo HTTPS.
==> 8. Activacion en nginx
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful

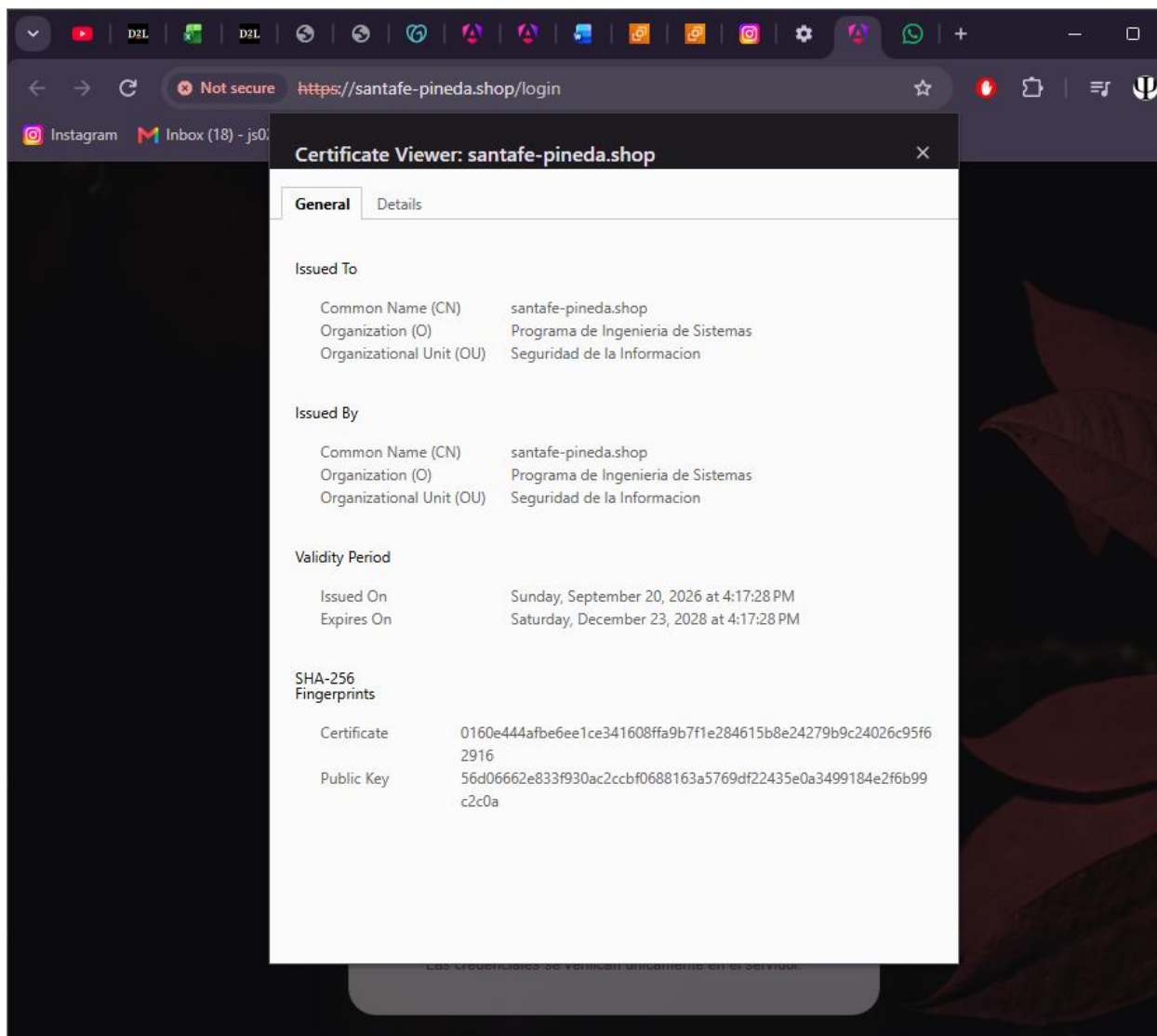
Archivos creados:
/etc/ssl/private/santafe-pineda.shop.key    llave privada (640 root:ssl-cert)
/etc/ssl/certs/santafe-pineda.shop.csr     solicitud de firma
/etc/ssl/certs/santafe-pineda.shop.crt     certificado autofirmado
/etc/ssl/private/dhparam.pem              parametros Diffie-Hellman

Archivo de nginx modificado:
/etc/nginx/sites-available/santafe-pineda.shop (ssl_certificate y ssl_certificate_key)
enlazado desde /etc/nginx/sites-enabled/santafe-pineda.shop

Evidencia 2 del informe:
- https://santafe-pineda.shop en Firefox -> pantalla de advertencia
  "Advertencia: riesgo probable de seguridad" con el codigo
  SEC_ERROR_UNKNOWN_ISSUER. Capturar tambien el detalle del certificado
  (Ver certificado -> Emitido por = el propio dominio).
- Prueba desde consola, sin aceptar la excepcion:
  openssl s_client -connect santafe-pineda.shop:443 -servername santafe-pineda.shop </dev/null |
  head -20
  Debe aparecer "verify error:num=18:self-signed certificate".
- SSL Server Test (ssllabs.com/ssltest) sobre este dominio dara calificacion T:
  el canal cifra bien, pero la identidad no la respalda nadie.

```





[Scan Another »](#)

```
# Options used in the renewal process
[renewalparams]
account = 3281920cdda7cde9b8315ebc1c7295f
pref_challs = http-01
authenticator = manual
server = https://acme-v02.api.letsencrypt.org/directory
key_type = ecdsa

subject=CN = santafe-pineda.shop
issuer=C = US, O = Let's Encrypt, CN = YE1
notBefore=Sep 20 21:17:20 2026 GMT
notAfter=Dec 19 21:17:23 2026 GMT
X509v3 Subject Alternative Name:
    DNS:santafe-pineda.shop, DNS:www.santafe-pineda.shop
=> 7: Activacion manual en nginx
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful

Certificado activo. Archivos implicados:

CREADOS POR CERTBOT
/etc/letsencrypt/live/santafe-pineda.shop/fullchain.pem    certificado + cadena intermedia
/etc/letsencrypt/live/santafe-pineda.shop/privkey.pem     llave privada
/etc/letsencrypt/live/santafe-pineda.shop/chain.pem       cadena intermedia
/etc/letsencrypt/live/santafe-pineda.shop/cert.pem        certificado del sitio
/etc/letsencrypt/renewal/santafe-pineda.shop.conf         parametros de renovacion

MODIFICADO A MANO POR USTED
/etc/nginx/sites-available/santafe-pineda.shop            ssl_certificate, ssl_certificate_key,
                                                           ssl_trusted_certificate, HSTS y el
                                                           bloque 301 del puerto 80
/etc/nginx/snippets/tls-params.conf                       protocolos y suites
(copia de seguridad de la etapa anterior en santafe-pineda.shop.autofirmado.bak)

Evidencia 3 del informe:
bash deploy/scripts/04-verificar-tls.sh

CERTIFICADO COMODIN (punto 6 del enunciado, opcional)
El reto http-01 no sirve para comodines; hay que usar dns-01, tambien manual:

    sudo certbot certonly --manual --preferred-challenges dns \
        -d santafe-pineda.shop -d '*.santafe-pineda.shop' --agree-tos --js9207367@gmail.com

Certbot pedira crear un registro TXT llamado _acme-challenge.santafe-pineda.shop con el
valor que indique. Se crea en el panel DNS del registrador, se comprueba con

    dig +short TXT _acme-challenge.santafe-pineda.shop @1.1.1.1

y solo cuando ya responde se pulsa Enter.
ubuntu@ip-172-31-22-193:~/Taller-de-Cifrado-Clasico/Taller/Taller-cifrado$
```

```
git pull
sudo rm -f /var/www/acme/.well-known/acme-challenge/NOMBRE_ARCHIVO
remote: Enumerating objects: 19, done.
remote: Counting objects: 100% (19/19), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 10 (delta 7), reused 10 (delta 7), pack-reused 0 (from 0)
Unpacking objects: 100% (10/10), 3.16 KiB | 402.00 KiB/s, done.
From https://github.com/Semiramyz/Taller-de-Cifrado-Clasico
d227ef9..7d5a261 main -> origin/main
Updating d227ef9..7d5a261
Fast-forward
  Taller/Taller-cifrado/deploy/scripts/02-letsencrypt-manual.sh | 8 +---
  Taller/Taller-cifrado/deploy/scripts/crear-reto-acme.sh       | 81 +++++
  Taller/Taller-cifrado/docs/PASO-A-PASO-AWS.md                | 12 +---
  3 files changed, 80 insertions(+), 21 deletions(-)
ubuntu@ip-172-31-22-193:~/Taller-de-Cifrado-Clasico/Taller/Taller-cifrado$ sudo bash deploy/scripts/crear-reto-acme.sh TQp6mJp1ROIiTXt0SsUK5snofMpimudZpee2oqfeIPU.P4fM-pvageWj9ptcqA24YgwYqU79P1Pw09M0yc07QeA
Archivo creado:
-rw-r--r-- 1 www-data www-data 87 Sep 20 22:14 /var/www/acme/.well-known/acme-challenge/TQp6mJp1ROIiTXt0SsUK5snofMpimudZpee2oqfeIPU
Contenido:
TQp6mJp1ROIiTXt0SsUK5snofMpimudZpee2oqfeIPU.P4fM-pvageWj9ptcqA24YgwYqU79P1Pw09M0yc07QeA
Comprobacion contra nginx (debe devolver 200 y el mismo contenido):
santafe-pineda.shop -> HTTP 200
www.santafe-pineda.shop -> HTTP 200

Retos presentes ahora mismo (deben acumularse, no sustituirse):
TQp6mJp1ROIiTXt0SsUK5snofMpimudZpee2oqfeIPU

Si el codigo es 200, vuelva a la sesion de certbot y pulse Enter.
ubuntu@ip-172-31-22-193:~/Taller-de-Cifrado-Clasico/Taller/Taller-cifrado$ sudo bash deploy/scripts/crear-reto-acme.sh aQJhmzSL1QjMj9f8Zfgo9c7d9En5Bo_uAt94e089oQs.P4fM-pvageWj9ptcqA24YgwYqU79P1Pw09M0yc07QeA
Archivo creado:
-rw-r--r-- 1 www-data www-data 87 Sep 20 22:15 /var/www/acme/.well-known/acme-challenge/aQJhmzSL1QjMj9f8Zfgo9c7d9En5Bo_uAt94e089oQs
Contenido:
aQJhmzSL1QjMj9f8Zfgo9c7d9En5Bo_uAt94e089oQs.P4fM-pvageWj9ptcqA24YgwYqU79P1Pw09M0yc07QeA
Comprobacion contra nginx (debe devolver 200 y el mismo contenido):
santafe-pineda.shop -> HTTP 200
www.santafe-pineda.shop -> HTTP 200

Retos presentes ahora mismo (deben acumularse, no sustituirse):
TQp6mJp1ROIiTXt0SsUK5snofMpimudZpee2oqfeIPU
aQJhmzSL1QjMj9f8Zfgo9c7d9En5Bo_uAt94e089oQs

Si el codigo es 200, vuelva a la sesion de certbot y pulse Enter.
ubuntu@ip-172-31-22-193:~/Taller-de-Cifrado-Clasico/Taller/Taller-cifrado$
```

```
ubuntu@ip:172-31-22-193: ~/ x + v
a:PKEY: id-ecPublicKey, 384 (bit); sigalg: ecdsa-with-SHA384
v:NotBefore: Sep  3 00:00:00 2025 GMT; NotAfter: Sep  2 23:59:59 2028 GMT
2 s:C = US, O = ISRG, CN = Root YE
i:C = US, O = Internet Security Research Group, CN = ISRG Root X2
a:PKEY: id-ecPublicKey, 384 (bit); sigalg: ecdsa-with-SHA384
v:NotBefore: May 13 00:00:00 2026 GMT; NotAfter: Sep  2 23:59:59 2032 GMT
3 s:C = US, O = Internet Security Research Group, CN = ISRG Root X2
i:C = US, O = Internet Security Research Group, CN = ISRG Root X1
a:PKEY: id-ecPublicKey, 384 (bit); sigalg: RSA-SHA256
v:NotBefore: May 13 00:00:00 2026 GMT; NotAfter: Sep  2 23:59:59 2032 GMT
---

----- 6. Emisor, titular y vigencia -----
subject=CN = santafe-pineda.shop
issuer=C = US, O = Let's Encrypt, CN = YE1
notBefore=Sep 20 21:17:24 2026 GMT
notAfter=Dec 19 21:17:23 2026 GMT
X509v3 Subject Alternative Name:
    DNS:santafe-pineda.shop, DNS:www.santafe-pineda.shop

----- 7. Resultado de la validacion -----
Verification: OK
Verify return code: 0 (ok)
Con Let's Encrypt debe decir: Verify return code: 0 (ok)

----- 8. Protocolos antiguos: deben ser RECHAZADOS -----
tls1: rechazado (correcto)
tls1_1: rechazado (correcto)

----- 9. Protocolos vigentes: deben ser ACEPTADOS -----
tls1_2: aceptado (correcto)
tls1_3: aceptado (correcto)

----- 10. Pruebas externas (requisito 4) -----
Plugin de Firefox o pagina de Qualys:
    https://www.ssllabs.com/ssltest/analyze.html?d=santafe-pineda.shop&hideResults=on&latest
Ejecutarlo DOS veces y guardar ambas capturas:
    - con el certificado autofirmado -> calificacion T (trust failure)
    - con el de Let's Encrypt -> calificacion A o A+
En Firefox, el candado -> Conexion segura -> Mas informacion muestra el
emisor, la vigencia y la version de TLS negociada.
```

y solo cuando ya responde se pulsa Enter.

```
ubuntu@ip-172-31-22-193:~/Taller-de-Cifrado-Clasico/Taller/Taller-cifrado$ sudo ls -l /etc/letsencrypt
/live/santafe-pineda.shop/
sudo cat /etc/letsencrypt/renewal/santafe-pineda.shop.conf
total 4
-rw-r--r-- 1 root root 692 Sep 20 22:15 README
lrwxrwxrwx 1 root root 43 Sep 20 22:15 cert.pem -> ../../archive/santafe-pineda.shop/cert1.pem
lrwxrwxrwx 1 root root 44 Sep 20 22:15 chain.pem -> ../../archive/santafe-pineda.shop/chain1.pem
lrwxrwxrwx 1 root root 48 Sep 20 22:15 fullchain.pem -> ../../archive/santafe-pineda.shop/fullchain1.
pem
lrwxrwxrwx 1 root root 46 Sep 20 22:15 privkey.pem -> ../../archive/santafe-pineda.shop/privkey1.pem
# renew_before_expiry = 30 days
version = 2.9.0
archive_dir = /etc/letsencrypt/archive/santafe-pineda.shop
cert = /etc/letsencrypt/live/santafe-pineda.shop/cert.pem
privkey = /etc/letsencrypt/live/santafe-pineda.shop/privkey.pem
chain = /etc/letsencrypt/live/santafe-pineda.shop/chain.pem
fullchain = /etc/letsencrypt/live/santafe-pineda.shop/fullchain.pem

# Options used in the renewal process
[renewalparams]
account = 3241920cdda7c4e9b8315ebc1c7295f
pref_challs = http-01,
authenticator = manual
server = https://acme-v02.api.letsencrypt.org/directory
key_type = ecdsa
ubuntu@ip-172-31-22-193:~/Taller-de-Cifrado-Clasico/Taller/Taller-cifrado$ |
```

```
ubuntu@ip-172-31-22-193:~/Taller-de-Cifrado-Clasico/Taller/Taller-cifrado$ sudo grep -n -A2 "ssl_certi
ficate\\|Strict-Transport\\|return 301" \
/etc/nginx/sites-available/santafe-pineda.shop
48:         return 301 https://$host$request_uri;
49-     }
50-}
--
63:     ssl_certificate          /etc/letsencrypt/live/santafe-pineda.shop/fullchain.pem;
64:     ssl_certificate_key      /etc/letsencrypt/live/santafe-pineda.shop/privkey.pem;
65:     ssl_trusted_certificate  /etc/letsencrypt/live/santafe-pineda.shop/chain.pem;
66-
--
82:     add_header Strict-Transport-Security "max-age=63072000; includeSubDomains" always;
83-
84:     # X-Content-Type-Options, X-Frame-Options y Referrer-Policy las emite la
ubuntu@ip-172-31-22-193:~/Taller-de-Cifrado-Clasico/Taller/Taller-cifrado$ |
```

```

----- 5. Cadena de certificacion completa -----
Certificate chain
 0 s:CN = santafe-pineda.shop
  i:C = US, O = Let's Encrypt, CN = YE1
  a:PKEY: id-ecPublicKey, 256 (bit); sigalg: ecdsa-with-SHA384
  v:NotBefore: Sep 20 21:17:24 2026 GMT; NotAfter: Dec 19 21:17:23 2026 GMT
 1 s:C = US, O = Let's Encrypt, CN = YE1
  i:C = US, O = ISRG, CN = Root YE
  a:PKEY: id-ecPublicKey, 384 (bit); sigalg: ecdsa-with-SHA384
  v:NotBefore: Sep 3 00:00:00 2025 GMT; NotAfter: Sep 2 23:59:59 2028 GMT
 2 s:C = US, O = ISRG, CN = Root YE
  i:C = US, O = Internet Security Research Group, CN = ISRG Root X2
  a:PKEY: id-ecPublicKey, 384 (bit); sigalg: ecdsa-with-SHA384
  v:NotBefore: May 13 00:00:00 2026 GMT; NotAfter: Sep 2 23:59:59 2032 GMT
 3 s:C = US, O = Internet Security Research Group, CN = ISRG Root X2
  i:C = US, O = Internet Security Research Group, CN = ISRG Root X1
  a:PKEY: id-ecPublicKey, 384 (bit); sigalg: RSA-SHA256
  v:NotBefore: May 13 00:00:00 2026 GMT; NotAfter: Sep 2 23:59:59 2032 GMT
---

----- 6. Emisor, titular y vigencia -----
subject=CN = santafe-pineda.shop
issuer=C = US, O = Let's Encrypt, CN = YE1
notBefore=Sep 20 21:17:24 2026 GMT
notAfter=Dec 19 21:17:23 2026 GMT
X509v3 Subject Alternative Name:
    DNS:santafe-pineda.shop, DNS:www.santafe-pineda.shop

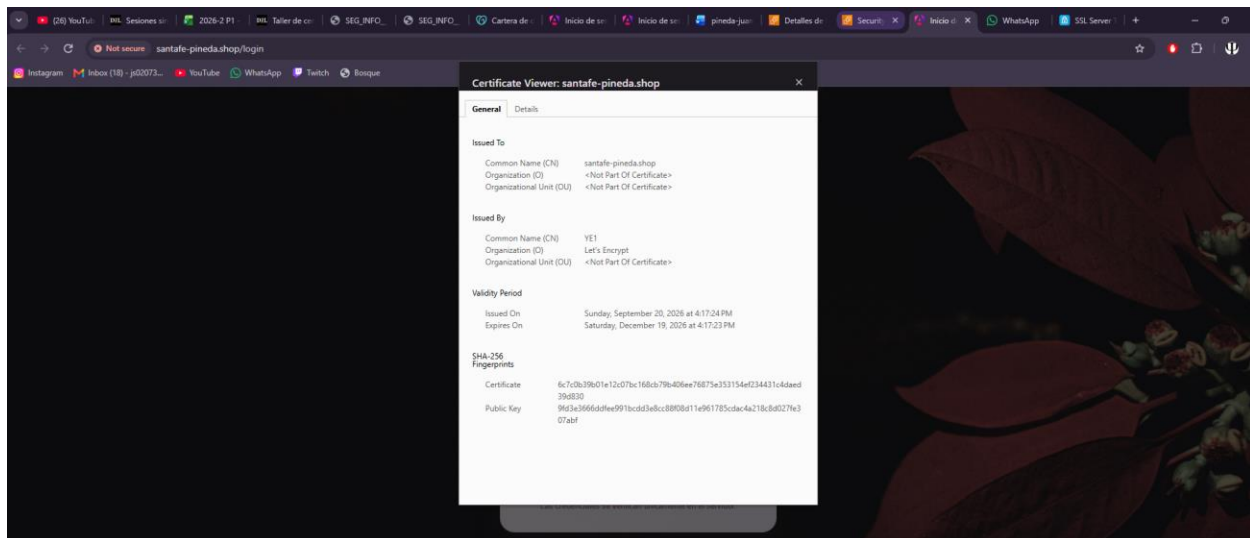
----- 7. Resultado de la validacion -----
Verification: OK
Verify return code: 0 (ok)
Con Let's Encrypt debe decir: Verify return code: 0 (ok)

----- 8. Protocolos antiguos: deben ser RECHAZADOS -----
tls1: rechazado (correcto)
tls1_1: rechazado (correcto)

----- 9. Protocolos vigentes: deben ser ACEPTADOS -----
tls1_2: aceptado (correcto)
tls1_3: aceptado (correcto)

----- 10. Pruebas externas (requisito 4) -----
Plugin de Firefox o pagina de Qualys:
  https://www.ssllabs.com/ssltest/analyze.html?d=santafe-pineda.shop&hideResults=on&latest
Ejecutarlo DOS veces y guardar ambas capturas:
  - con el certificado autofirmado -> calificacion T (trust failure)
  - con el de Let's Encrypt -> calificacion A o A+
En Firefox, el candado -> Conexion segura -> Mas informacion muestra el
emisor, la vigencia y la version de TLS negociada.
ubuntu@ip-172-31-22-193:~/Taller-de-Cifrado-Clasico/Taller/Taller-cifrado$

```



4.3. Paso 3: la autofirma

El tercer paso firma la solicitud, y lo hace con la misma llave privada que la generó. De ahí el nombre: el certificado se firma a sí mismo. En un certificado emitido por una Autoridad Certificadora este paso lo ejecutaría la Autoridad con **su** llave, y sería su firma —y no la nuestra— la que el navegador reconocería.

Se añaden además las extensiones X.509v3, y entre ellas `subjectAltName` merece una nota. Desde 2017 los navegadores ignoran por completo el campo CN al verificar el nombre del sitio y atienden sólo a esta extensión. Un certificado sin ella se rechaza con el error `ERR_CERT_COMMON_NAME_INVALID` aunque el nombre común sea correcto, y es una causa habitual de confusión al seguir manuales antiguos.

```
cat > /tmp/santafe-pineda.shop.ext <<EXT
basicConstraints = critical, CA:FALSE
keyUsage         = critical, digitalSignature, keyEncipherment
extendedKeyUsage = serverAuth
subjectAltName   = DNS:santafe-pineda.shop, DNS:www.santafe-pineda.shop
EXT

openssl x509 -req -days 825 -sha256 \
    -in      /etc/pki/tls/certs/santafe-pineda.shop.csr \
    -signkey /etc/pki/tls/private/santafe-pineda.shop.key \
    -extfile /tmp/santafe-pineda.shop.ext \
    -out     /etc/pki/tls/certs/santafe-pineda.shop.crt
```

La vigencia se fijó en 825 días porque ése es el máximo que los navegadores aceptan para un certificado de servidor; valores mayores se rechazan con independencia de lo que diga el certificado.

4.4. Archivos creados y activación en nginx

Archivo	Contenido
/etc/pki/tls/private/santafe-pineda.shop.key	Llave privada, permisos 600

Archivo	Contenido
/etc/pki/tls/certs/santafe-pineda.shop.csr	Solicitud de firma
/etc/pki/tls/certs/santafe-pineda.shop.crt	Certificado autofirmado
/etc/pki/tls/private/dhparam.pem	Parámetros Diffie-Hellman propios

La activación consiste en añadir un bloque que escuche en el puerto 443 y declare dónde están esos dos archivos. Son literalmente dos directivas: son las mismas que cambiarán en la etapa 3, apuntando entonces a las rutas de Let's Encrypt.

```
/etc/nginx/conf.d/santafe-pineda.shop.conf (etapa 2, fragmento)
server {
    listen      443 ssl;
    listen      [::]:443 ssl;
    http2       on;
    server_name santafe-pineda.shop www.santafe-pineda.shop;

    ssl_certificate      /etc/pki/tls/certs/santafe-pineda.shop.crt;
    ssl_certificate_key  /etc/pki/tls/private/santafe-pineda.shop.key;

    location / {
        include          /etc/nginx/fragmentos/proxy-node.conf;
        proxy_pass       http://127.0.0.1:4000;
    }
}
```

Una decisión deliberada: en esta etapa no se activó la cabecera HSTS, aunque técnicamente era posible. HSTS ordena al navegador negarse a hablar en claro con el dominio durante el plazo indicado, y combinarlo con un certificado que el navegador no puede validar dejaría el sitio inaccesible: el usuario ni siquiera tendría la opción de aceptar la excepción. HSTS se introduce en la etapa 3, cuando el certificado ya es de confianza pública.

4.5. La problemática asociada

Al visitar el sitio, Firefox interrumpe la navegación con la pantalla "Advertencia: riesgo probable de seguridad" y el código SEC_ERROR_UNKNOWN_ISSUER. El texto del error es preciso y merece leerse con atención: el emisor es desconocido. No dice que el cifrado sea débil, ni que el certificado esté caducado, ni que el nombre no corresponda. Dice que quien firmó ese certificado no figura en el almacén de Autoridades de confianza del navegador.

La misma conclusión se obtiene desde la consola, sin necesidad de aceptar ninguna excepción:

```
openssl s_client -connect santafe-pineda.shop:443 -servername santafe-pineda.shop
</dev/null

depth=0 C=CO, ST=Santander, ..., CN=santafe-pineda.shop
```

```
verify error:num=18:self-signed certificate
verify return:1
```

En el visor de certificados de Firefox el síntoma se ve de un vistazo: los campos "Emitido para" y "Emitido por" contienen exactamente el mismo nombre. Esa coincidencia es la firma visual del problema. Un certificado legítimo siempre tiene un emisor distinto del titular, y esa cadena se puede seguir hacia arriba hasta una raíz que el navegador ya conoce de fábrica.

SSL Labs resume la situación con la calificación **T**, de trust failure. Es una calificación especial, no una nota baja dentro de la escala: significa que la evaluación se detuvo porque el certificado no es de confianza. La herramienta llega incluso a indicar cuál habría sido la nota si el problema de confianza se resolviera, lo que confirma lo dicho al principio: la calidad criptográfica del canal era correcta; lo que faltaba era la identidad.

El problema práctico es que ningún visitante debería acostumbrarse a pulsar "Aceptar el riesgo y continuar". Esa costumbre es justamente lo que un ataque de intermediario necesita para pasar inadvertido, porque el certificado falso del atacante produce en el navegador una advertencia idéntica a la que el usuario ya aprendió a ignorar. Por eso el certificado autofirmado sirve para entornos internos y pruebas, pero no para un sitio público.

INSERTAR CAPTURA · 04-openssl-generacion.png

Salida completa de los tres pasos de OpenSSL: generación de la llave, la CSR y la autofirma.

INSERTAR CAPTURA · 05-autofirmado-advertencia.png

Pantalla de Firefox "Advertencia: riesgo probable de seguridad" con el código SEC_ERROR_UNKNOWN_ISSUER.

INSERTAR CAPTURA · 06-autofirmado-detalle.png

Visor de certificados de Firefox: "Emitido para" y "Emitido por" con el mismo nombre.

INSERTAR CAPTURA · 07-autofirmado-ssllabs.png

SSL Labs con calificación T sobre santafe-pineda.shop.

5. Etapa 3: certificado de Let's Encrypt emitido e instalado a mano

Let's Encrypt es una Autoridad Certificadora gratuita cuya raíz, ISRG Root X1, está incluida en el almacén de confianza de todos los navegadores actuales. Sus certificados ofrecen menos garantías que uno de validación extendida —sólo acreditan el control sobre el dominio, no la existencia legal de una organización—, pero resuelven por completo el problema que dejó abierto el certificado autofirmado: la identidad del servidor queda respaldada por un tercero en quien el navegador ya confía.

5.1. Qué significa hacerlo manualmente

El enunciado exige que tanto la creación como la activación del certificado sean manuales. Traducido a decisiones concretas sobre las herramientas, eso significó tres cosas, y las tres se tomaron de forma explícita:

Decisión	Lo que se hizo	Lo que se evitó
Emisión	certbot certonly	certbot --nginx, que además instala solo
Reto ACME	--manual: el token se escribe a mano	--webroot, que lo deposita certbot
Activación	Se editaron a mano las directivas ssl_certificate	Que certbot reescriba la configuración
Renovación	Repetición manual cada 90 días	certbot-renew.timer y el cron de certbot

La diferencia entre certonly y el instalador automático es el punto central. Con certonly certbot hace una sola cosa: obtiene el certificado y lo deja en el disco. No abre ningún archivo de configuración de nginx ni escribe una sola línea en él. El instalador automático, en cambio, detecta el bloque server correspondiente, le inserta las directivas, crea por su cuenta el bloque de redirección y deja unos comentarios "managed by Certbot" para marcar lo que ha tocado. El resultado funciona, pero el administrador no llega a saber qué archivos se modificaron ni dónde quedó cada pieza, que es precisamente lo que el ejercicio quiere que se aprenda.

Para reforzar esa separación se instaló el paquete certbot sin el complemento python3-certbot-nginx. Sin ese paquete la opción --nginx ni siquiera existe en la línea de órdenes, de modo que la instalación automática queda descartada por construcción y no por disciplina.

5.2. Preparativos

Antes de emitir se comprobó que el registro A del dominio apunta a la dirección IP pública del servidor y que los puertos 80 y 443 están abiertos en firewalld. También se preparó la carpeta donde se depositará el token del reto.

```
dnf install -y epel-release
dnf install -y certbot          # sin python3-certbot-nginx, a proposito

install -d -m 755 /var/www/acme/.well-known/acme-challenge
chown -R nginx:nginx /var/www/acme
```

```
restorecon -Rv /var/www/acme # SELinux: etiqueta la carpeta como contenido web
```

La orden `restorecon` no es un detalle menor en Rocky Linux. SELinux está activo por omisión y una carpeta creada fuera de las rutas habituales no recibe la etiqueta adecuada, con lo que `nginx` no puede leerla y el reto falla con un 403 que desde fuera parece un problema de permisos corrientes. Por la misma razón fue necesario activar el booleano `httpd_can_network_connect`, sin el cual SELinux impide que `nginx` abra la conexión hacia el puerto 4000 y el sitio responde 502.

5.3. La emisión: el reto http-01 resuelto a mano

La orden de emisión declara el modo manual y el tipo de reto. Se incluyen el dominio y su variante con `www`, de modo que un único certificado cubra ambos nombres mediante la extensión `subjectAltName`.

```
certbot certonly \
  --manual \
  --preferred-challenges http \
  --agree-tos --no-eff-email -m js0207367@gmail.com \
  -d santafe-pineda.shop \
  -d www.santafe-pineda.shop
```

En este punto `certbot` se detiene y espera. Muestra un texto con la forma siguiente y no continúa hasta que se pulsa `Enter`:

```
Create a file containing just this data:

QmFzZTY0VG9rZW4tZWplbXBsbw.9xK3vN8pQ2wLmR7tYuIoP4sD6fGhJ1kZ

And make it available on your web server at this URL:

http://santafe-pineda.shop/.well-known/acme-challenge/QmFzZTY0VG9rZW4tZWplbXBsbw

Press Enter to Continue
```

El contenido tiene dos partes separadas por un punto: el token que propone el servidor de Let's Encrypt y la huella de la clave de la cuenta ACME. Esa segunda mitad es lo que impide que un tercero se apropie del reto, porque sólo quien posee la clave de la cuenta puede componer el valor correcto.

Sin cerrar esa sesión se abre una segunda conexión SSH y se crea el archivo. Es el momento que da sentido a todo el ejercicio: queda a la vista que el reto es, literalmente, un archivo de texto en una ruta concreta del servidor.

```
# En una SEGUNDA sesion SSH
install -d -m 755 /var/www/acme/.well-known/acme-challenge
echo -n 'QmFzZTY0VG9rZW4tZWplbXBsbw.9xK3vN8pQ2wLmR7tYuIoP4sD6fGhJ1kZ' \
  > /var/www/acme/.well-known/acme-challenge/QmFzZTY0VG9rZW4tZWplbXBsbw
chown -R nginx:nginx /var/www/acme && restorecon -Rv /var/www/acme
```

```
# Comprobacion antes de continuar
curl http://santafe-pineda.shop/.well-known/acme-
challenge/QmFzZTY0VG9rZW4tZWplbXBsbnw
```

El echo `-n` es importante: el archivo no debe llevar salto de línea final, porque el servidor de validación compara el contenido byte a byte. Sólo cuando el `curl` devuelve el valor esperado se vuelve a la primera sesión y se pulsa Enter. Let's Encrypt solicita entonces esa URL desde varios puntos de internet y, si el contenido coincide, emite el certificado.

Para que esa petición llegue fue necesario prever una excepción en la configuración: el bloque del puerto 80 redirige todo a HTTPS **salvo** la ruta del reto. El protocolo ACME exige que el token se lea por HTTP en el puerto 80, y por esa ruta sólo se entregan archivos de texto estáticos, nunca contenido de la aplicación. El apartado 7 vuelve sobre esta excepción y sobre cómo eliminarla si se prefiere.

5.4. Archivos que crea certbot y dónde quedan

Superada la validación, certbot deposita el certificado y muestra las rutas. Saber qué hay en cada archivo es justamente el objetivo declarado del ejercicio.

Ruta	Qué contiene
/etc/letsencrypt/live/santafe-pineda.shop/fullchain.pem	Certificado del sitio más la intermedia. Es el que se declara en nginx.
/etc/letsencrypt/live/santafe-pineda.shop/privkey.pem	Llave privada generada por certbot.
/etc/letsencrypt/live/santafe-pineda.shop/chain.pem	Sólo la intermedia. Se usa para la validación OCSP.
/etc/letsencrypt/live/santafe-pineda.shop/cert.pem	Sólo el certificado del sitio, sin la cadena.
/etc/letsencrypt/archive/santafe-pineda.shop/	Los archivos reales, numerados por emisión. live/ son enlaces simbólicos a los más recientes.
/etc/letsencrypt/renewal/santafe-pineda.shop.conf	Los parámetros con que se emitió, que se reutilizan al renovar.
/etc/letsencrypt/accounts/	La clave de la cuenta ACME, con la que se firma cada solicitud.

Por qué fullchain.pem y no cert.pem. Es el error más frecuente al instalar un certificado a mano. El navegador necesita poder encadenar el certificado del sitio con la Autoridad raíz que ya conoce, y entre ambos hay una intermedia. Si se declara `cert.pem`, que contiene sólo el certificado del sitio, la cadena queda rota: algunos navegadores la completan por su cuenta porque tienen la intermedia en caché de visitas anteriores a otros sitios, lo que hace que el fallo parezca intermitente, y SSL Labs lo reporta sin ambigüedad como "Chain issues: incomplete". La distinción entre estructura del directorio y enlaces

simbólicos también importa: las directivas deben apuntar siempre a `live/`, nunca a `archive/`, porque son los enlaces de `live/` los que se actualizan al renovar.

5.5. Activación manual en nginx

Con los archivos ya en el disco, la instalación propiamente dicha se hizo editando a mano la configuración del servidor. Este es el segundo tramo manual del ejercicio y consiste en tres directivas.

```
/etc/nginx/conf.d/santafe-pineda.shop.conf (etapa 3, fragmento del bloque 443)
server {
    listen      443 ssl default_server;
    listen      [::]:443 ssl default_server;
    http2       on;
    server_name santafe-pineda.shop www.santafe-pineda.shop;

    ssl_certificate      /etc/letsencrypt/live/santafe-
pineda.shop/fullchain.pem;
    ssl_certificate_key  /etc/letsencrypt/live/santafe-pineda.shop/privkey.pem;
    ssl_trusted_certificate /etc/letsencrypt/live/santafe-pineda.shop/chain.pem;

    add_header Strict-Transport-Security "max-age=63072000; includeSubDomains"
always;

    location / {
        include      /etc/nginx/fragmentos/proxy-node.conf;
        proxy_pass    http://127.0.0.1:4000;
    }
}
```

```
nginx -t                # comprueba la sintaxis antes de aplicar
systemctl reload nginx  # recarga sin cortar las conexiones en curso
```

Se emplea `reload` y no `restart` porque el primero relee la configuración y los certificados levantando procesos de trabajo nuevos mientras los antiguos terminan de atender lo que tenían entre manos, sin cortar ninguna conexión.

5.6. Desactivación de la renovación automática

El enunciado plantea dos modalidades para Let's Encrypt: la renovación manual cada noventa días y la programación de una tarea automática. Se optó por la primera, coherente con el resto del ejercicio, lo que obligó a desactivar la automatización que el paquete de `certbot` instala por su cuenta.

```
systemctl disable --now certbot-renew.timer
systemctl mask certbot-renew.timer
rm -f /etc/cron.d/certbot

systemctl list-timers --all | grep -i certbot  # sin resultados
```

Conviene añadir que, en este caso, la renovación automática no habría funcionado aunque se hubiera dejado activa. Un certificado emitido con `--manual` y reto `http-01` no puede renovarse sin intervención por diseño, porque certbot necesita que alguien vuelva a colocar el token. Es un rasgo del modo elegido, no una limitación imprevista: la renovación se describe en el apartado 10.

INSERTAR CAPTURA · 09-certbot-pausa.png

Certbot detenido, mostrando el contenido del archivo del reto y la URL donde debe quedar disponible. Es la evidencia central de que la emisión fue manual.

INSERTAR CAPTURA · 10-token-creado.png

Segunda sesión SSH: creación del archivo del reto y el curl que devuelve el token.

INSERTAR CAPTURA · 11-certbot-exito.png

"Successfully received certificate" con las rutas de `/etc/letsencrypt/live/`.

INSERTAR CAPTURA · 12-archivos-letsencrypt.png

Listado de `/etc/letsencrypt/live/santafe-pineda.shop/` mostrando los enlaces simbólicos, y el contenido de `renewal/santafe-pineda.shop.conf`.

INSERTAR CAPTURA · 13-nginx-editado.png

El archivo `conf.d/santafe-pineda.shop.conf` con las directivas `ssl_certificate` escritas a mano, sin comentarios "managed by Certbot".

INSERTAR CAPTURA · 14-renovacion-desactivada.png

`systemctl list-timers --all` sin ningún temporizador de certbot.

6. Certificado comodín mediante el reto DNS-01

El enunciado recuerda que pueden emitirse certificados comodín o para varios dominios. El certificado de la etapa 3 ya cubre dos nombres mediante `subjectAltName`, que es la forma habitual de resolver el caso de varios dominios. El comodín, en cambio, exige un mecanismo de validación distinto.

Un certificado para `*.santafe-pineda.shop` no puede emitirse con el reto `http-01`, y la razón es lógica: el reto consiste en colocar un archivo en un servidor concreto, y un comodín afirma algo sobre todos los subdominios posibles, incluidos los que aún no existen. Demostrar control sobre un nombre no demuestra control sobre todos. Let's Encrypt exige por ello el reto `dns-01`, que se resuelve creando un registro `TXT` en la zona del dominio y prueba el control sobre la zona entera.

```
certbot certonly --manual --preferred-challenges dns \
  -d santafe-pineda.shop -d '*.santafe-pineda.shop' \
  --agree-tos -m js0207367@gmail.com

# Certbot pide crear este registro y esperar a que propague:
# _acme-challenge.santafe-pineda.shop  TXT  "valor-que-indique-certbot"

dig +short TXT _acme-challenge.santafe-pineda.shop @1.1.1.1
```

El registro se crea en el panel DNS del registrador y no debe pulsarse Enter hasta que la consulta lo devuelva desde un resolutor público, porque la propagación tarda entre unos segundos y varios minutos según el proveedor. Este reto es también manual, con la ventaja añadida de que no necesita abrir nada en el puerto 80.

7. Redirección obligatoria a HTTPS

El segundo requisito del enunciado pide que quien entre por HTTP sea redirigido a HTTPS y que no se permitan accesos sin cifrar. Se resolvió en tres capas que se refuerzan entre sí.

7.1. La redirección en nginx

El bloque del puerto 80 no sirve ningún contenido: devuelve una redirección permanente. Se declara además como servidor predeterminado, de modo que también caen ahí las peticiones dirigidas a la dirección IP desnuda o a un nombre desconocido, y ninguna llega en claro a la aplicación por una puerta lateral.

```
server {
    listen      80 default_server;
    listen     [::]:80 default_server;
    server_name santafe-pineda.shop www.santafe-pineda.shop _;

    # Unica excepcion: el reto ACME, que el protocolo exige leer por HTTP.
    location ^~ /.well-known/acme-challenge/ {
        root    /var/www/acme;
    }
}
```

```

        default_type "text/plain";
        try_files $uri =404;
    }

    location / {
        return 301 https://$host$request_uri;
    }
}

```

Se devuelve un 301 y no un 302 porque la redirección es definitiva y el navegador debe recordarla. Se conserva `$request_uri` para que la ruta y los parámetros sobrevivan al salto y el usuario llegue a la página que pedía. La única excepción, ya comentada, es la carpeta del reto ACME; quien prefiera no dejar ninguna excepción puede emitir con el reto dns-01 del apartado 6 y suprimir ese bloque.

7.2. HSTS

La redirección tiene una ventana de vulnerabilidad inevitable: la primera petición en claro llega a viajar por la red antes de ser redirigida, y un atacante situado en el camino puede interceptarla. La cabecera `Strict-Transport-Security` cierra esa ventana para las visitas siguientes. Tras la primera respuesta, el navegador anota el dominio durante dos años y convierte por su cuenta cualquier enlace `http` a `https` **antes** de abrir la conexión, sin esperar a la redirección del servidor.

7.3. La aplicación

Por último, la propia aplicación redirige a HTTPS lo que llegue en claro. En el despliegue actual nunca debería dispararse, puesto que `nginx` ya lo impide; está como segunda línea de defensa por si alguna vez se publicara sin el proxy delante o alguien alcanzase el puerto 4000 directamente.

Con una salvedad que merece mencionarse: las peticiones que no son GET ni HEAD no se redirigen, sino que se rechazan con un 403. Redirigir un POST haría que el navegador reenviara el cuerpo —las credenciales del formulario— por un canal que ya se sabe inseguro. Es preferible cortar.

8. Endurecimiento de la configuración TLS

Los parámetros criptográficos se agruparon en un archivo aparte, en el contexto `http`, de modo que los hereden todos los bloques del servidor. Corresponden al perfil "intermediate" recomendado por Mozilla, que es el que conduce a la calificación A en SSL Labs.

```

/etc/nginx/conf.d/tls-parametros.conf
ssl_protocols TLSv1.2 TLSv1.3;
ssl_ciphers ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:...;
ssl_prefer_server_ciphers off;
ssl_dhparam /etc/pki/tls/private/dhparam.pem;
ssl_session_cache shared:MozSSL:10m;
ssl_session_timeout 1d;
ssl_session_tickets off;

```

- **TLS 1.2 y 1.3 únicamente.** SSLv3, TLS 1.0 y TLS 1.1 están rotos —POODLE y BEAST— y su mera presencia hace que SSL Labs limite la calificación.
- **Suites con ECDHE.** El intercambio efímero de claves proporciona forward secrecy: quien grabe hoy el tráfico cifrado y obtenga mañana la llave privada del servidor seguirá sin poder descifrarlo.
- **Tickets de sesión desactivados.** Los tickets sin rotación de clave anulan en la práctica esa propiedad, así que se prefiere la caché compartida.
- **OCSP stapling no se activó.** Let's Encrypt retiró su servicio OCSP y sus certificados ya no incluyen la URL del respondedor, con lo que la directiva sólo produciría un aviso en el registro. La revocación se atiende hoy por listas CRL y, sobre todo, por la vida corta de noventa días del certificado. Queda documentado y desactivado a propósito en la configuración.

9. Ajustes necesarios en la aplicación

Instalar el certificado en nginx no basta para que la aplicación se beneficie de él. Como el proxy le entrega las peticiones ya descifradas por el bucle local, la aplicación ve tráfico en claro procedente siempre de la misma dirección, y eso tiene dos consecuencias que habrían pasado inadvertidas sin una revisión.

La primera afecta a la cookie de sesión. Está marcada Secure, lo que significa que el navegador sólo la envía por HTTPS; pero si Express considera insegura la petición, la cookie no llega a emitirse y el inicio de sesión no funciona. La segunda afecta al control de fuerza bruta construido en el taller anterior: sin la configuración adecuada, req.ip devuelve siempre 127.0.0.1, la dirección del propio proxy, y los intentos fallidos de todo internet se contabilizan contra una única dirección. El bloqueo por IP dejaría de proteger y pasaría a ser un modo trivial de dejar fuera a todos los usuarios legítimos a la vez.

Ambas cosas se resuelven declarando a Express que hay un proxy de confianza delante, con lo que pasa a leer la cabecera X-Forwarded-Proto que nginx añade. Angular en modo SSR mantiene además su propia validación, independiente de la de Express, y exige declararle por separado qué cabeceras acepta y qué nombres de dominio considera válidos.

Archivo	Cambio introducido
src/server.ts	trust proxy en Express y trustProxyHeaders en Angular SSR
src/backend/middlewares/https.ts	Redirección a HTTPS y cabeceras de seguridad de la respuesta
src/backend/config.ts	Sección https leída del entorno
angular.json	security.allowedHosts con el dominio real
.env	COOKIE_SECURE=true, CONFIAR_EN_PROXY=1, FORZAR_HTTPS=true

El ajuste de allowedHosts merece destacarse porque su omisión produce un fallo desconcertante: Angular responde **400 Bad Request** a toda petición cuya cabecera Host no sea localhost. El sitio quedaría inaccesible en el dominio real con un certificado impecable, y el error no menciona el certificado por

ninguna parte. Es una defensa contra falsificación de la cabecera Host, y sencillamente hay que declararle el dominio de producción.

10. Verificación de los dos certificados

El cuarto requisito pide comprobar ambos certificados con el complemento de Firefox indicado o con la página de Qualys SSL Labs. Las pruebas se ejecutaron sobre el mismo dominio en los dos estados, y el contraste es el resultado más ilustrativo de toda la práctica.

Aspecto evaluado	Autofirmado	Let's Encrypt
Calificación de SSL Labs	T (trust failure)	A
Cadena de certificación	Incompleta: no llega a ninguna raíz	Completa hasta ISRG Root X1
Validación de openssl s_client	verify error:num=18	Verify return code: 0 (ok)
Comportamiento de Firefox	SEC_ERROR_UNKNOWN_ISSUER	Candado cerrado, "Verificado por: Let's Encrypt"
Protocolos aceptados	TLS 1.2 y 1.3	TLS 1.2 y 1.3
Forward secrecy	Sí	Sí
Vigencia	825 días	90 días

Las dos últimas filas antes de la vigencia son las que conviene leer con detenimiento, porque son idénticas: la calidad del canal cifrado no cambió entre una etapa y otra. Lo único que cambió fue quién respalda la identidad del servidor, y eso bastó para pasar de una calificación T a una A. Es la demostración más clara de que cifrado y autenticación son problemas distintos.

El certificado de noventa días, que a primera vista parece un inconveniente, es en realidad una decisión de seguridad deliberada de Let's Encrypt: reduce la ventana de aprovechamiento de una llave comprometida y obliga a que el proceso de renovación esté siempre engrasado, en lugar de convertirse en una operación excepcional que se hace mal una vez al año.

Las comprobaciones desde la consola se automatizaron en un guion que deja constancia escrita de cada prueba:

```
bash deploy/scripts/04-verificar-tls.sh | tee deploy/evidencias/verificacion.txt
curl -sSI http://santafe-pineda.shop/ -> 301 + Location: https://...
curl -sSI https://santafe-pineda.shop/ -> Strict-Transport-Security
presente
openssl s_client -connect santafe-pineda.shop:443 -> Verify return code: 0 (ok)
```

INSERTAR CAPTURA · 15-firefox-candado.png

Firefox sobre <https://santafe-pineda.shop/> con el candado cerrado y el panel de conexión segura indicando "Verificado por: Let's Encrypt".

INSERTAR CAPTURA · 16-redireccion-301.png

curl -I http://santafe-pineda.shop/ devolviendo 301 y la cabecera Location hacia https.

INSERTAR CAPTURA · 17-ssllabs-A.png

Informe de SSL Labs con calificación A o A+ y la cadena de certificación completa.

11. Renovación manual cada noventa días

Al haberse emitido el certificado con `--manual` y reto `http-01`, la renovación repite el mismo procedimiento: `certbot` vuelve a detenerse a pedir un token nuevo, que se coloca de la misma forma. La orden `certbot renew` no sirve por sí sola en este modo.

```
# Comprobar cuanto queda
openssl x509 -in /etc/letsencrypt/live/santafe-pineda.shop/cert.pem -noout -dates

# Renovar (se recomienda a partir de los 30 días restantes)
certbot certonly --manual --preferred-challenges http --force-renewal \
    -d santafe-pineda.shop -d www.santafe-pineda.shop

nginx -t && systemctl reload nginx
```

Después de renovar no hay que tocar la configuración de `nginx`. Las directivas apuntan a `live/`, cuyos enlaces simbólicos `certbot` ya ha hecho apuntar a los archivos nuevos; basta con recargar para que `nginx` los lea del disco. Si las directivas apuntaran a `archive/`, el sitio seguiría sirviendo el certificado antiguo hasta caducar, que es la razón de fondo por la que siempre se referencia `live/`.

12. Conclusiones

La práctica deja una conclusión principal que las tres etapas hacen difícil de olvidar: el cifrado y la autenticación de la identidad son problemas independientes, y un certificado SSL resuelve los dos a la vez sólo cuando lo respalda una Autoridad reconocida. El certificado autofirmado cifra exactamente igual de bien que el de Let's Encrypt —las suites, los protocolos y el forward secrecy fueron idénticos— y aun así resulta inservible para un sitio público, porque el usuario no tiene manera de distinguirlo del certificado falso de un atacante.

La segunda conclusión es de orden práctico y justifica la insistencia del enunciado en el procedimiento manual. Automatizar la instalación produce el mismo resultado visible con una fracción del esfuerzo, pero deja al administrador sin saber qué archivos existen ni cuáles se modificaron. Haberlo hecho a mano permite responder sin dudar dónde está la llave privada, por qué se declara `fullchain.pem` y no `cert.pem`, qué ocurre exactamente durante la validación del dominio y qué hay que tocar cuando algo falla. Ese conocimiento es el que sirve el día en que la renovación automática no funciona y el sitio queda con un certificado caducado.

La tercera conclusión surgió durante el trabajo y no estaba prevista: instalar el certificado en el servidor web no termina la tarea si la aplicación vive detrás de un proxy. La cookie de sesión marcada Secure, el registro correcto de la dirección del visitante y el propio control de fuerza bruta del taller anterior dependen de que la aplicación sepa que hay un terminador TLS delante. Un certificado perfecto sobre una aplicación mal informada produce un sitio que o no deja iniciar sesión, o bloquea a todos sus usuarios a la vez en cuanto alguien intenta un ataque de fuerza bruta.

Por último, el trabajo encaja con lo hecho en los talleres anteriores y lo completa. El cifrado clásico mostró cómo se protege un mensaje; el módulo de inicio de sesión protegió las credenciales almacenadas con BCrypt y frenó la fuerza bruta; esta práctica protege esas mismas credenciales mientras viajan. Las tres piezas son necesarias, y la ausencia de cualquiera de ellas deja sin efecto buena parte de las otras dos.

Anexo A. Mapa de archivos del servidor

Resumen de qué se modificó y dónde quedó cada cosa, que es el objetivo declarado del ejercicio.

<code>/etc/nginx/ conf.d/santafe-pineda.shop.conf conf.d/tls-parametros.conf fragmentos/proxy-node.conf</code>	ARCHIVO PRINCIPAL. Cambia en cada etapa. protocolos, suites, dhparam cabeceras hacia 127.0.0.1:4000
<code>/etc/pki/tls/ certs/santafe-pineda.shop.crt certs/santafe-pineda.shop.csr private/santafe-pineda.shop.key private/dhparam.pem</code>	CERTIFICADO AUTOFIRMADO certificado solicitud de firma llave privada (600) parametros Diffie-Hellman
<code>/etc/letsencrypt/ live/santafe-pineda.shop/ archive/santafe-pineda.shop/ renewal/santafe-pineda.shop.conf accounts/</code>	CERTIFICADO LET'S ENCRYPT enlaces: fullchain, privkey, chain, cert archivos reales, numerados parametros de la emision clave de la cuenta ACME
<code>/var/www/acme/.well-known/acme-challenge/ /etc/systemd/system/taller-cifrado.service</code>	token del reto, escrito a mano servicio de la aplicacion

Anexo B. Material entregado en el repositorio

Ruta en el repositorio	Contenido
deploy/README.md	Guía de despliegue y mapa de archivos
deploy/nginx/01-sin-ssl.conf	Configuración de la etapa 1
deploy/nginx/02-autofirmado.conf	Configuración de la etapa 2
deploy/nginx/03-letsencrypt.conf	Configuración de la etapa 3, con redirección y HSTS
deploy/nginx/tls-parametros.conf	Protocolos, suites y parámetros de sesión
deploy/scripts/00-preparar-servidor.sh	Instalación, servicio systemd, firewalld y SELinux
deploy/scripts/01-certificado-autofirmado.sh	Generación y activación del autofirmado
deploy/scripts/02-letsencrypt-manual.sh	Emisión manual con certbot certonly --manual
deploy/scripts/crear-reto-acme.sh	Colocación a mano del token del reto
deploy/scripts/03-renovar-letsencrypt-manual.sh	Renovación manual cada 90 días
deploy/scripts/04-verificar-tls.sh	Batería de comprobaciones para las evidencias
deploy/systemd/taller-cifrado.service	Unidad de servicio de la aplicación

Ruta en el repositorio	Contenido
deploy/evidencias/	Capturas de pantalla y salida de las verificaciones